

CHUNKING



The Decision That Everything
Else Inherits From

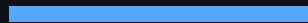
Building an Agentic RAG System

Part 1



The Problem

"Most RAG tutorials treat chunking as a footnote."



Your embedding model, retrieval precision, re-ranker, and generator quality are ALL downstream of how you split documents.

Get chunking wrong
and no model swap recovers it.



The Naive Approach

Fixed-Size Chunking

```
chunks = [text[i:i+512]
          for i in range(
              0, len(text), 512
          )]
```

What happens:

- Table split mid-row -- the next chunk has no heading or context
- 3 topics in 1 chunk -- the embedding vector is an average of all three



Topic Dilution

Why averaging kills retrieval

Basic
Scraping

PDF
Parsing

Scrape
Options



ONE averaged embedding vector

"Ask about PDF parsing specifically
and this chunk scores mediocre --
the answer is averaged out."



Header-Driven Chunking

"A markdown heading is a semantic
boundary declared by the author.
More reliable than any token heuristic."

Each heading level = a topic boundary:

= Page topic

= Section topic

= Sub-section topic

One section = one chunk = one topic
= one clean embedding vector



The Breadcrumb Prefix

```
Section: Advanced Scraping Guide
> Scrape options
> Formats
```

Prepended to every chunk before
embedding and indexing.

- Richer embedding vectors: model sees the full topic path, not just body text
- Better BM25 tokens: high-signal terms like "scraping", "options", "formats" appear in the document representation



Smart Merging

Handling pure-header chunks

BEFORE

```
Chunk A: ["## Scrape options"]  
        (empty -- just a header!)  
  
Chunk B: ["### Formats..."]  
        (the actual content)
```

AFTER

```
Chunk B: [  
  "## Scrape options",  
  "### Formats...",  
  "The formats array..."  
]
```

No wasted index slots.
No content-free embeddings.



The Results

Metric	Before	After
HR@1	~33%	66.67%
HR@5	~58%	91.67%
MRR@5	~0.38	0.78

Same embedding model.
Same search algorithm.
Only the chunk boundaries changed.



What's Next

Three known failure modes to solve

Level 3

Giant Sections

Token overflow -- embedding truncates the tail silently.

Fix: overlapping sliding window within the section.

Level 4

Cross-Section Answers

Answer spans two sections -- neither chunk retrieves well.

Fix: inter-section overlap + parent expansion.

Level 5

No Headers / Code

Flat files produce one giant chunk. Code blocks embed poorly.

Fix: paragraph fallback + language-tagged code chunks.





**Chunking quality
sets the ceiling for
everything downstream.**



"You cannot retrieve what
wasn't cleanly embedded."

Follow for Part 2:

**BPE vs Whitespace Tokenization
for BM25**

